contents

preface

acknowledgments

about this book

about the authors

About the cover illustration

1 **Welcome to the cloud, Spring**

1.1 The evolution towards a microservices architecture

N-tier architecture

What's a monolithic architecture?

What's a microservice?

Why change the way we build applications?

1.2 Microservices with Spring

1.3 What are we building?

1.4 What is this book about?

What you'll learn in this book

Why is this book relevant to you?

1.5 Cloud and microservice-based applications

Building a microservice with Spring Boot

What exactly is cloud computing?

Why the cloud and microservices?

1.6 Microservices are more than writing the code

1.7 Core microservice development pattern

1.8 Microservice routing patterns

1.9 Microservice client resiliency

1.10 Microservice security patterns

1.11 Microservice logging and tracing patterns

1.12 Application metrics pattern

1.13 Microservice build/deployment patterns

2.1 What is Spring Cloud?

Spring Cloud Config

Spring Cloud Service Discovery

Spring Cloud LoadBalancer and Resilience4j

Spring Cloud API Gateway

Spring Cloud Stream

Spring Cloud Sleuth

Spring Cloud Security

2.2 Spring Cloud by example

2.3 How to build a cloud-native microservice

Codebase

Dependencies

Config

Backing services

Build, release, run

Processes

Port binding

Concurrency

Disposability

Dev/prod parity

Logs

Admin processes

2.4 Making sure our examples are relevant

2.5 Building a microservice with Spring Boot and Java

Setting up the environment

Getting started with the skeleton project

Booting your Spring Boot application: Writing the bootstrap class

3 Building microservices with Spring Boot

3.1 The architect's story: Designing the microservice architecture

Decomposing the business problem

Establishing service granularity

Defining the service interfaces

3.2 When not to use microservices

Complexity when building distributed systems

Server or container sprawl

Application type

Data transactions and consistency

3.3 The developer's tale: Building a microservice with Spring Boot and Java

Building the doorway into the microservice: The Spring Boot controller

Adding internationalization into the licensing service

Implementing Spring HATEOAS to display related links

3.4 The DevOps story: Building for the rigors of runtime

Service assembly: Packaging and deploying your microservices

Service bootstrapping: Managing configuration of your microservices

Service registration and discovery: How clients communicate with your microservices

Communicating a microservice's health

3.5 Pulling the perspectives together

4 Welcome to Docker

4.1 Containers or virtual machines?

4.2 What is Docker?

4.3 Dockerfiles

4.4 Docker Compose

4.5 Integrating Docker with our microservices

Building the Docker Image

Creating Docker images with Spring Boot

Launching the services with Docker Compose

5 Controlling your configuration with the Spring Cloud Configuration Server

5.1 On managing configuration (and complexity)

Your configuration management architecture

Implementation choices

5.2 Building our Spring Cloud Configuration Server

Setting up the Spring Cloud Config bootstrap class

Using the Spring Cloud Config Server with a filesystem

Setting up the configuration files for a service

5.3 Integrating Spring Cloud Config with a Spring Boot client

Setting up the licensing service Spring Cloud Config Service dependencies

Configuring the licensing service to use Spring Cloud Config

Wiring in a data source using Spring Cloud Config Server

Directly reading properties using @ConfigurationProperties

Refreshing your properties using Spring Cloud Config Server

Using Spring Cloud Configuration Server with Git

Integrating Vault with the Spring Cloud Config service

Vault UI

5.4 Protecting sensitive configuration information

Setting up a symmetric encryption key

Encrypting and decrypting a property

5.5 Closing thoughts

6 On service discovery

6.1 Where's my service?

6.2 Service discovery in the cloud

The architecture of service discovery

Service discovery in action using Spring and Netflix Eureka

6.3 Building our Spring Eureka service

6.4 Registering services with Spring Eureka

Eureka's REST API

Eureka dashboard

6.5 Using service discovery to look up a service

Looking up service instances with Spring Discovery Client

Invoking services with a Load Balancer–aware Spring REST template

Invoking services with Netflix Feign client

7 When bad things happen: Resiliency patterns with Spring Cloud and Resilience4j

7.1 What are client-side resiliency patterns?

Client-side load balancing

Circuit breaker 180Fallback processing

Bulkheads

7.2 Why client resiliency matters

7.3 Implementing Resilience4j

7.4 Setting up the licensing service to use Spring Cloud and Resilience4j

7.5 Implementing a circuit breaker

Adding the circuit breaker to the organization service

Customizing the circuit breaker

7.6 Fallback processing

7.7 Implementing the bulkhead pattern

7.8 Implementing the retry pattern

7.9 Implementing the rate limiter pattern

7.10 ThreadLocal and Resilience4j

8 Service routing with Spring Cloud Gateway

8.1 What is a service gateway?

8.2 Introducing Spring Cloud Gateway

Setting up the Spring Boot gateway project

Configuring the Spring Cloud Gateway to communicate with Eureka

8.3 Configuring routes in Spring Cloud Gateway

Automated mapping of routes via service discovery

Manually mapping routes using service discovery

Dynamically reloading route configuration

8.4 The real power of Spring Cloud Gateway: Predicate and Filter Factories

Built-in Predicate Factories

Built-in Filter Factories

Custom filters

8.5 Building the pre-filter

8.6 Using the correlation ID in the services

UserContextFilter: Intercepting the incoming HTTP request

UserContext: Making the HTTP headers easily accessible to the service

Custom RestTemplate and UserContextInterceptor: Ensuring that the correlation ID gets propagated

8.7 Building a post-filter receiving correlation ID

9 Securing your microservices

9.1 What is OAuth2?

9.2 Introduction to Keycloak

9.3 Starting small: Using Spring and Keycloak to protect a single endpoint

Adding Keycloak to Docker

Setting up Keycloak

Registering a client application

Configuring O-stock users

Authenticating our O-stock users

9.4 Protecting the organization service using Keycloak

Adding the Spring Security and Keycloak JARs to the individual services

Configuring the service to point to our Keycloak server

Defining who and what can access the service

Propagating the access token

Parsing a custom field in a JWT

9.5 Some closing thoughts on microservice security

Use HTTPS secure sockets layer (SSL) for all service communication

Use a service gateway to access your microservices

Zone your services into a public API and private API

Limit the attack surface of your microservices by locking down unneeded network ports

10 Event-driven architecture with Spring Cloud Stream

10.1 The case for messaging, EDA, and microservices

Using a synchronous request-response approach to communicate state change

Using messaging to communicate state changes between services

Downsides of a messaging architecture

10.2 Introducing Spring Cloud Stream

10.3 Writing a simple message producer and consumer

Configuring Apache Kafka and Redis in Docker

Writing the message producer in the organization service

Writing the message consumer in the licensing service

Seeing the message service in action

10.4 A Spring Cloud Stream use case: Distributed caching

Using Redis to cache lookups

Defining custom channels

11 Distributed tracing with Spring Cloud Sleuth and Zipkin

11.1 Spring Cloud Sleuth and the correlation ID

Adding Spring Cloud Sleuth to licensing and organization

Anatomy of a Spring Cloud Sleuth trace

11.2 Log aggregation and Spring Cloud Sleuth

A Spring Cloud Sleuth/ELK Stack implementation in action

Configuring Logback in our services

Defining and running ELK Stack applications in Docker

Configuring Kibana

Searching for Spring Cloud Sleuth trace IDs in Kibana

Adding the correlation ID to the HTTP response with Spring Cloud Gateway

11.3 Distributed tracing with Zipkin

Setting up the Spring Cloud Sleuth and Zipkin dependencies

Configuring the services to point to Zipkin

Configuring a Zipkin server

Setting tracing levels

Using Zipkin to trace transactions

Visualizing a more complex transaction

Capturing messaging traces

Adding custom spans

12 Deploying your microservices

12.1 The architecture of a build/deployment pipeline

12.2 Setting up O-stock's core infrastructure in the cloud

Creating the PostgreSQL database using Amazon RDS

Creating the Redis cluster in Amazon

12.3 Beyond the infrastructure: Deploying O-stock and ELK

Creating an EC2 with ELK

Deploying the ELK Stack in the EC2 instance

Creating an EKS cluster

12.4 Your build/deployment pipeline in action

12.5 Creating our build/deploy pipeline

Setting up GitHub

Enabling our services to build in Jenkins

Understanding and generating the pipeline script

Creating the Kubernetes pipeline scripts

12.6 Closing thoughts on the build/deployment pipeline

appendix A. Microservices architecture best practices

appendix B. OAuth2 grant types

appendix C. Monitoring your microservices

index

Previous chapter
brief contents

Next chapter
front matter